

SmartShelf

Smart Retail Inventory & First-Expiry-First-Out (FEFO) Management System

TEAM REPORT - PART 2 of 4 | Team Member 2: [Your Name]

Your role on the project: Core engine owner

What YOU present and defend: FEFO picking, stock batches, purchase receiving, inventory ledger and the sales flow - including how we guarantee no overselling.

How to use this part

Read your sections below until you can walk someone through them without notes. Then practise the Q&A cues at the end - they mirror what a panel typically asks. If you do not know a detail outside your part, say so and hand it to the right teammate.

You must be able to explain

- What FEFO is and how it is implemented per sale.
- Why the stock update must be atomic (concurrency).
- What a batch is and how expiry classification works.
- The purchase/PO receiving flow into batches.
- The movement ledger types (IN/OUT/ADJUST/TRANSFER/WASTAGE).
- How a sale keeps batch-accurate traceability.

1. FEFO - the core idea

First-Expired-First-Out: when stock is taken, always take from the batch that expires first. SmartShelf sorts live batches by expiry date ascending, skips expired ones (they go to waste), allocates from the oldest, and tops up from the next when a batch runs out.

2. How a sale deducts stock (implementation)

- POST /api/sales receives items and payment details.
- For each product the engine walks its batches earliest-expiry first.
- Each deduction is one guarded atomic statement: UPDATE stock_batches SET quantity = quantity - n WHERE id = ? AND quantity >= n.
- If the UPDATE changes zero rows the batch is empty - the whole sale rolls back and returns 400.
- Sales and PostgreSQL additionally lock rows (SELECT ... FOR UPDATE) during allocation.

3. Concurrency: the no-oversell guarantee

Two cashiers could read quantity=1 and both sell the last unit. The guarded UPDATE makes that impossible: only one transaction changes the row, the other matches zero rows and is refused.

Proof test - tests/test_concurrency.py fires 20 parallel 1-unit sales: all 20 succeed, stock ends at 0 (never negative), 20 OUT movements are written, and an oversized sale is cleanly refused.

4. Batches and expiry classification

- A batch is one lot: product + batch code + quantity, cost price, expiry date, supplier.
- Expiry scan labels each batch OK / EXPIRING_SOON / EXPIRED within a 30-day window.
- Zeroed batches leave sellable stock but their history stays in the ledger.

5. Purchase orders and receiving (stock in)

- A manager creates a PO with line items (product, qty, unit cost, batch code, expiry).
- Receiving calls add_stock: create or top up the batch, raise product stock, append an IN movement.
- Partial receipts are tracked (received_quantity); a full receipt marks the PO RECEIVED.

6. Inventory control and the movement ledger

Locations model the shop floor; transfers move stock between them; adjustments fix counts after stocktakes. Every change writes a ledger line of type IN / OUT / ADJUST / TRANSFER / WASTAGE - the immutable audit trail of every unit.

7. Sales flow and traceability

- Each sale line becomes one SaleItem per allocated batch - the invoice shows exactly which expiry-dated batch was sold.
- Totals: subtotal - discount + tax = total; a Payment row records the method.
- An invoice number (INV-XXXX) is generated and the sale is audited in one commit.

8. Endpoints you should be able to quote

METHOD	PATH	ACCESS	PURPOSE
POST	/api/sales	STAFF+	Record sale with FEFO deduction + payment
POST	/api/purchases//receive	MANAGER+	Turn ordered lines into batches
POST	/api/inventory/receive	MANAGER+	Direct stock receipt
POST	/api/inventory/adjust	MANAGER+	Correct a physical count
GET	/api/batches	Any user	All batches with expiry
GET	/api/fefo	Any user	FEFO pick order per product
GET	/api/inventory/movements	Any user	Movement ledger

9. Likely questions and answer cues

Q. What does FEFO mean and where do you apply it?

A. First-Expired-First-Out - sell the earliest-expiring batch first. Applied at every sale and transfer; expired batches are excluded to waste.

Q. How does the engine choose batches?

A. It sorts live batches by expiry date ascending, then allocates the requested quantity from the oldest batch, topping up from the next whenever one is exhausted.

Q. What happens if there is not enough stock?

A. The guarded UPDATE matches fewer rows than needed; the route rolls back the entire sale and returns a 400 error - there is never a partial or negative deduction.

Q. How do you prevent two cashiers overselling the last unit?

A. A single atomic UPDATE .. WHERE quantity >= n. Only one concurrent transaction wins each row; the other sees zero rows and is refused.

Q. What is a 'batch' exactly?

A. One dated lot of one product, identified by product + batch code, with quantity, landed cost, expiry date and the supplier who sent it.

Q. What movement types exist?

A. IN (received), OUT (sold), ADJUST (stocktake correction), TRANSFER (between locations), WASTAGE (written off) - all recorded in inventory_movements.

Q. How is traceability guaranteed on an invoice?

A. Each SaleItem stores batch_id and quantity, so any invoice line can be reconciled to the actual expiry-dated unit sold.

Q. How do purchase orders turn into stock?

A. The PO lists expected lines; receiving walks them, calling add_stock per line to create/top up the batch and write an IN movement; a fully received PO becomes RECEIVED.